Installation (first time only):

python -m venv .venv

.\.venv\Scripts\Activate.ps1
python -m pip install pandas numpy matplotlib seaborn scikit-learn scipy

EXPERIMENT 1: Boston Housing - Linear Regression

Code :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset from CSV file
df = pd.read_csv("housing.csv")

# Separate features and target
X = df[['RM', 'LSTAT', 'PTRATIO']]
y = df['MEDV']

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train the regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict values
y_pred = model.predict(X_test)
```

```
# Evaluate the model
print("Actual values:", list(y_test))
print("Predicted values:", y_pred.round(2))
print("Mean Squared Error:", round(mean_squared_error(y_test, y_pred), 2))
print("R2 Score:", round(r2_score(y_test, y_pred), 2))

# Predict for a new house
new_house = pd.DataFrame(
    [[6.5, 5.0, 15.5]],
    columns=['RM', 'LSTAT', 'PTRATIO']
)

print("Predicted MEDV:", round(model.predict(new_house)[0], 2))
```

Dataset: housing.csv

Create a file named 1housing.csv with the following 20 rows:

```
RM,LSTAT,PTRATIO,MEDV
6.575,4.98,15.3,24.0
6.421,9.14,15.4,21.9
7.185,4.03,15.6,34.7
6.968,2.94,15.6,33.4
6.629,5.33,16.0,36.2
6.302,7.88,15.6,28.7
6.114,12.26,15.6,22.9
5.617,17.27,16.0,27.1
6.006,11.74,15.6,16.5
5.933,13.27,16.0,18.9
5.794,15.27,15.6,15.0
6.495,5.52,15.6,32.0
6.861,6.52,15.6,29.0
```

6.579,8.12,15.6,24.5

5.405,20.62,16.0,10.2

6.487,3.13,15.6,31.5

5.852,13.0,16.0,13.3

6.128,10.45,15.6,19.6

6.727,4.21,15.6,27.8

5.950,14.80,15.6,12.0

How to Create the Dataset File

1. Open a text editor (Notepad, VS Code, etc.)
2. Copy the CSV content above (including header)
3. Save as 'housing.csv' in the same directory as your Python script

How to Run the Experiment

Command to start:
python your_script_name.py

Installation (first time only):
python -m pip install pandas numpy matplotlib seaborn scikit-learn scipy

EXPERIMENT 2: Titanic - Logistic Regression

Overview

Predict whether a Titanic passenger survived (1) or not (0) using Logistic Regression. Features: Pclass, Sex, Age, SibSp, Parch, Fare, Embarked.

Code :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("titanic.csv")

# Convert categorical columns into numeric columns
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True,
dtype=int)

# Handle missing values
df = df.dropna()

# Separate features and target
X = df.drop('Survived', axis=1)
y = df['Survived']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# Train Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Predict survival
y_pred = model.predict(X_test)

# Evaluate model
print("Actual values:", list(y_test))
print("Predicted values:", list(y_pred))
print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
# Create confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])

# Plot confusion matrix as a heatmap
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Did not survive', 'Survived'],
    yticklabels=['Did not survive', 'Survived']
)

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Titanic Survival Confusion Matrix')
plt.show()
```

```
# Predict for a new passenger
new_passenger = pd.DataFrame([{
    'Pclass': 3,
    'Age': 25,
    'SibSp': 0,
    'Parch': 0,
    'Fare': 7.05,
    'Sex_male': 1,
    'Embarked_Q': 0,
    'Embarked_S': 1
}], columns=X.columns)

prediction = model.predict(new_passenger)[0]

print("Survival Prediction:", "Survived" if prediction == 1 else "Did not survive")
```

Dataset: 2titanic.csv (set USE_SAMPLE=False to keep the dataset name this)

Create a file named 2titanic.csv with the following 20 rows:

Pclass,Sex,Age,SibSp,Parch,Fare,Embarked,Survived

1,female,38.0,1,0,71.2833,C,1

1,male,47.0,1,0,52.8625,S,1

3,female,26.0,0,0,7.925,S,1

1,female,35.0,1,0,53.1,S,1

3,male,35.0,0,0,8.05,S,0

3,male,25.0,0,0,7.05,S,0

1,female,58.0,0,1,146.5208,C,1

3,female,5.0,4,2,29.125,S,1

3,female,27.0,0,0,7.925,S,1

2,male,14.0,1,0,30.0708,C,1

1,female,4.0,1,1,120.0,S,1

1,male,58.0,0,0,26.55,S,1

3,male,20.0,0,0,8.05,S,0

3,female,39.0,1,5,31.275,S,1

2,male,14.0,0,0,21.0,S,1

1,female,55.0,0,0,51.8625,S,1

3,male,2.0,3,1,21.075,S,1

1,male,9.0,3,2,263.0,S,1

3,female,38.0,1,5,31.3875,S,1

2,female,45.0,0,0,13.0,S,1

How to Create the Dataset File

1. Open a text editor2. Copy the CSV content above (including header)3. Save as '2titanic.csv' in the same directory as your Python script

How to Run the Experiment

Command to start:
python your_script_name.py

EXPERIMENT 3: Adult Census - Decision Tree

Overview

Predict income level (<=50K or >50K) using Decision Tree Classifier. Includes hyperparameter tuning with max_depth using 5-fold cross-validation.

Code :

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Load dataset
df = pd.read_csv("adult.csv")

# Separate features and target
X = df.drop("income", axis=1)
y = df["income"]

# Convert categorical data into numeric data
X = pd.get_dummies(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train Decision Tree model
model = DecisionTreeClassifier(random_state=42, max_depth=4)
model.fit(X_train, y_train)

# Predict income
y_pred = model.predict(X_test)

# Evaluate model
print("Actual values:", list(y_test))
print("Predicted values:", list(y_pred))
```

```
print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
print("Confusion Matrix:\n", confusion_matrix(
    y_test, y_pred, labels=model.classes_
))
print("\nClassification Report:\n", classification_report(
    y_test, y_pred, zero_division=0
))


plt.figure(figsize=(15, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True
)
plt.show()
```

Dataset: 3adult.csv

Create a file named 3adult.csv with 20 rows (simplified, 8 features):

age,workclass,education,marital-status,occupation,relationship,race,hours-per-week,income

39,State-gov,Bachelors,Never-married,Adm-clerical,Not-in-family,White,40,<=50K

50,Self-emp-inc,Bachelors,Married-civ-spouse,Exec-managerial,Husband,White,13,>50K

38,Private,HS-grad,Divorced,Handlers-cleaners,Not-in-family,White,40,<=50K

53,Private,11th,Married-civ-spouse,Handlers-cleaners,Husband,Black,40,<=50K

28,Private,Bachelors,Married-civ-spouse,Prof-specialty,Wife,Black,40,>50K

37,Federal-gov,HS-grad,Married-civ-spouse,Protective-serv,Husband,White,40,>50K

49,Private,9th,Married-spouse-absent,Other-service,Not-in-family,Black,16,<=50K

52,Self-emp-not-inc,HS-grad,Married-civ-spouse,Exec-managerial,Husband,White,40,>50K

31,Private,Masters,Never-married,Prof-specialty,Not-in-family,White,50,>50K

42,Private,Bachelors,Married-civ-spouse,Exec-managerial,Husband,White,40,>50K

37,Private,Some-college,Married-civ-spouse,Exec-managerial,Husband,Black,80,>50K

30,State-gov,Bachelors,Married-civ-spouse,Prof-specialty,Husband,Asian-Pac-Islander,40,>50K

23,Private,Bachelors,Never-married,Adm-clerical,Own-child,White,30,<=50K

40,Private,Assoc-acdm,Married-civ-spouse,Sales,Husband,White,40,>50K

58,Private,HS-grad,Widowed,Other-service,Not-in-family,Black,40,<=50K

35,Private,HS-grad,Married-civ-spouse,Craft-repair,Husband,White,40,<=50K

48,Self-emp-inc,Bachelors,Married-civ-spouse,Exec-managerial,Husband,White,60,>50K

36,Private,HS-grad,Married-civ-spouse,Machine-op-inspct,Husband,White,40,<=50K

29,Private,HS-grad,Never-married,Laborer,Own-child,White,40,<=50K

52,Private,HS-grad,Married-civ-spouse,Exec-managerial,Husband,White,40,>50K

How to Create the Dataset File

1. Open a text editor2. Copy the CSV content above (including header)3. Save as '3adult.csv' in the same directory as your Python script

How to Run the Experiment

Command to start:

python your_script_name.py

EXPERIMENT 4: Adult Census - Random Forest

Overview

Compare Random Forest with Decision Tree classifier for income prediction. Analyzes feature importance and model performance across both algorithms.

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.tree import plot_tree


# Load dataset
df = pd.read_csv("adult.csv")


# Separate features and target
X = df.drop("income", axis=1)
y = df["income"]
```

```
# Convert categorical data into numeric data
X = pd.get_dummies(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train Random Forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predict income
y_pred = model.predict(X_test)

# Evaluate model
print("Actual values:", list(y_test))
print("Predicted values:", list(y_pred))
print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))

cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
print("Confusion Matrix:\n", cm)

print("\nClassification Report:\n",
      classification_report(y_test, y_pred, zero_division=0))

# Visualize confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=model.classes_,
```

```
    yticklabels=model.classes_
)
plt.xlabel("Predicted Income")
plt.ylabel("Actual Income")
plt.title("Random Forest Confusion Matrix")
plt.tight_layout()
plt.show()

# Visualize one tree from the Random Forest
plt.figure(figsize=(16, 8))
plot_tree(
    model.estimators_[0],
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)
plt.title("Random Forest - One Decision Tree")
plt.tight_layout()
plt.show()
```

Dataset: adult.csv (same as Experiment 3)

Use the same adult.csv file created in Experiment 3.

How to Run the Experiment

Command to start:
python your_script_name.py

EXPERIMENT 5: Adult Census - AdaBoost

Overview

Predict income using AdaBoost Classifier (100 weak stumps). Analyzes performance and feature importance, shows how boosting improves accuracy iteratively.

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.tree import plot_tree


# Load dataset
df = pd.read_csv("adult.csv")


# Separate features and target
X = df.drop("income", axis=1)
y = df["income"]
```

```
# Convert categorical data into numeric data
X = pd.get_dummies(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train AdaBoost model
model = AdaBoostClassifier(n_estimators=50, random_state=42)
model.fit(X_train, y_train)

# Predict income
y_pred = model.predict(X_test)

# Evaluate model
print("Actual values:", list(y_test))
print("Predicted values:", list(y_pred))
print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))

cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
print("Confusion Matrix:\n", cm)

print("\nClassification Report:\n",
      classification_report(y_test, y_pred, zero_division=0))

# Visualize confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=model.classes_,
```

```
    yticklabels=model.classes_
)
plt.xlabel("Predicted Income")
plt.ylabel("Actual Income")
plt.title("AdaBoost Confusion Matrix")
plt.tight_layout()
plt.show()

# Visualize the first weak learner
plt.figure(figsize=(12, 6))
plot_tree(
    model.estimators_[0],
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True,
    max_depth=2
)
plt.title("AdaBoost - First Weak Learner")
plt.tight_layout()
plt.show()
```

Dataset: adult.csv (same as Experiment 3)

Use the same adult.csv file created in Experiment 3.

How to Run the Experiment

Command to start:
python your_script_name.py

EXPERIMENT 6: Wholesale Customers - Hierarchical Clustering

Overview

Group customers by spending patterns using Hierarchical Clustering (Ward linkage). Identifies 3 distinct customer segments based on product category spending.

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from scipy.spatial.distance import pdist


# Load dataset
df = pd.read_csv("customers.csv")


# Select spending features
X = df.drop(columns=["Channel", "Region"])


# Standardize the data
X_scaled = StandardScaler().fit_transform(X)
```

```
# Calculate Euclidean distances and Ward linkage
Z = linkage(X_scaled, method="ward", metric="euclidean")

# Plot dendrogram
plt.figure(figsize=(10, 5))
dendrogram(Z)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Customer Index")
plt.ylabel("Euclidean Distance")
plt.show()

# Create 3 customer clusters
df["Cluster"] = fcluster(Z, t=3, criterion="maxclust")

print("\nCustomer Clusters:")
print(df[["Fresh", "Milk", "Grocery", "Cluster"]])

# Visualize clusters
sns.scatterplot(
    data=df, x="Fresh", y="Milk",
    hue="Cluster", palette="Set1", s=100
)
plt.title("Wholesale Customer Clusters")
plt.show()
```

Dataset: customers.csv

Create a file named 6customers.csv with 20 customer rows:

Channel,Region,Fresh,Milk,Grocery,Frozen,Detergents_Paper,Delicassen

1,3,12669,9656,7561,214,2674,13382

1,3,7057,10039,9687,1490,8441,64042

1,3,6353,8808,7684,2405,1867,26131

1,3,13265,1196,4221,6404,507,17882

1,3,22615,5410,7198,3915,1777,51852

1,3,9413,8259,5126,666,1795,14512

1,3,12126,3199,6975,480,7837,37922

1,3,7579,4956,9426,1669,3321,25661

1,3,5963,3648,6192,425,1716,7502

1,3,6006,11093,18881,1159,7425,20982

1,3,3366,5403,12974,4400,5977,17441

1,3,17873,3957,5735,415,7709,112742

1,3,5899,11026,8772,645,6305,8252

1,3,1077,3480,6900,141,1271,10352

1,3,5263,28926,1361,3584,38669,14531

1,3,16591,13650,9540,1625,4981,88142

1,3,6970,3885,12121,971,7534,40501

1,3,7775,24127,7169,1260,4015,78722

1,3,9740,23127,3321,1500,9833,113052

1,3,32717,16784,13722,5927,11606,2871

How to Create the Dataset File

1. Open a text editor2. Copy the CSV content above (including header)3. Save as 'customers.csv' in the same directory as your Python script

How to Run the Experiment

Command to start:

python your_script_name.py